

Advanced Computer Networks LAB

Spring Semester 2026

Assignment 3

Name – Soham Chakraborty

Roll No – 22CS02002

Part 1 –

- 1. Problem:** servers/server: error while loading shared libraries: libavdevice.so.58: cannot open shared object file: No such file or directory
Solution: sudo apt install libavdevice-dev
- 2. Problem:** servers/server: error while loading shared libraries: libgtk-x11-2.0.so.0: cannot open shared object file: No such file or directory
Solution: sudo apt install libgtk2.0-0
- 3. Problem:** Determining how the data sent by server was encoded.
Solution: Since the server code was not provided to us. I want to know the various sections of the header/metadata were divided. Like in the previous assignment I separated the parts of header via a special symbol which was '|'. Later I found out that in the given server code since all sections were consuming a memory of exact power of 2. They were separated by a fixed bit offset in a fixed order. [*Binary Serialization*]
- 4. Problem:** Lowering the -d (--delay-between-frames) to 5 makes the program/client crash after sending a certain number of frames.
Solution: This is because the server is sending frames so fast that client can't keep up with decoding and displaying the frame and then moving on to the next one. In my code I have **waitKey(5)** i.e. I want for 5ms for escape key. Delay by server has to be higher than at least this value, because otherwise frames received by client would be incomplete or

multiple in between frames are skipped, resulting in segmentation fault. Also, it is noticed that lowering to around 10 makes the video run very fast around 3 times faster.

Thus, I believe that `--delay-between-frames` effectively controls the speed of video.

- 5. Problem:** Increasing `-d` (`--delay-between-frames`) to 100 makes the program/client very laggy.

Solution: This is because the server is sending frames extremely slowly and there is no way to improve this from the client end. Video runs 3 times slower as opposed to previous case.

However, there is a lower chance of multiple packets being sent one on top of another, so reliability in this case is very high (at the cost of performance).

Part 2 –

- 1. Problem:** Sending “REQ frame” to server gives wrong frame.

Solution: If I send “REQ 0” the server sends “frame48” instead of sending 0. However, when I send “REQ \0”, the server sends “frame0”. This means that I had to first compress the **`uint32_t frameIndex`** to a 4-byte character array and then send the request to server (discovered with the help of ChatGPT).

However, this was nowhere mentioned in the Problem Statement.

- 2. Problem:** Empty/Incorrect sections in received Frame Metadata.

Solution: Size of Frame Metadata as per Assignment pdf which is **11-bytes** is not matching with the server sent size of **16-bytes**. In this case, changing all variables types to **`uint32_t`** fixed the issue and also matches

the server size (I don't know why it didn't have any issues while doing the previous Assignment).

- 3. Problem:** Client getting infinitely stuck waiting for client sendto() after requesting a certain frame when using flag -c (--continuous-packet-drop).

Solution: *This is just a hypothesis since I don't have server code.*

I am assuming that whenever Client sends request for a certain wrong frameIndex if server receives the request while it is in the middle of sending some other frame then that request is ignored.

This was later fixed by adding a timeout to requestfrom() of 100ms. So whenever any response to "REQ i" doesn't come from server within 100ms. The client sends the request again. If repeated requests are sent by Client, then one of them eventually is received by server.

Another alternative to solve this is to set a very high delay say 1second or more after each failed request. This however, reduces the Video fps to < 1 , as the wait time and buffer is too high.

- 4. Problem:** Server sending frame0 after frame255.

Solution: Most likely it is due to the fact that my encoding method is different from how it is encoded and decoded in the server side. That's why at 256 frame number i.e. Starting at 2Byte frame_id it doesn't work. However, without knowing how server encoded and decodes the messages further improvement can't be done.

- 5. Problem:** Even with the above fix (3), if using flag -c then around frame 250, the video becomes very laggy to practically paused.

Solution: *Once again, this is just a hypothesis since I don't have server code.*

Consider the case, where the frame size to be sent by server is very large, and client already previously sent many requests to server for a particular frame. If the server processes all frame requests one by one and keeps sending the same large frame multiple times, it takes a lot of time. And after all that processing the frame sent by server becomes useless. In the meantime while waiting the required frame client sends multiple

requests for that frame and when it finally gets the desired frame, multiple previous requests to server for that frame is still pending. This results in a positive feedback loop and latency keeps increasing from then onwards.

One solution to this would be from the server side. Instead of processing all pending requests, only send frame from the last request. Also, whenever a frame request comes while another frame is in mid-transmission stop that transmission and start sending the newly requested frame. However, these are server side modifications and I can't resolve it without the source code. This should in theory remove most if not all of buffer observed.

6. Problem: Setting `--packet-drop-prob 0.9` or some other high value also introduces a very high lag.

Solution: Similar to what described in Problem 4 setting a high packet drop rate magnifies latency but much that what should have been expected. Because the positive feedback loop in this case just magnifies much more quickly. Changing some portion from server code I believe should drastically reduce the latency in this case too.

Part 3 –

There was no such problem faced while building server for Part 3, because most of it is covered in Part 1 and Part 2. Also, I had previously built 2 multiplayer online games based on Socket Programming in C++ (using TCP instead of UDP) [[SAHAS-Arcade](#) and [Soccer-Stars](#)], so there was no major problem as such.

1. **Problem:** *sizeof(Struct)* giving incorrect size due to Byte Alignment.

Solution: Add *#pragma pack(push,1)* before struct declaration and *#pragma pack(pop)* after declared. This keeps 1Byte alignment and avoid memory address issues.

2. **Problem:** Unable to connect from *client.exe* (Windows version) to my UDP server.

Solution: I am using Windows 11 and using WSL 2 (Ubuntu 22.04) to run my *server3.cpp* code. Instead of using *client.exe (Windows)*, doing - *./client (Linux Version)* is able to connect to my server.

3. **Irrelevant Problem:** Holding down spacebar does not keep the player jumping. Generally in video games, holding down spacebar or any other action key the player keeps doing it until released. However, in this case it only jumps once and then stops jumping.